

韦易笑C++笑话集

作者:韦易笑

原帖地址:https://www.zhihu.com/question/30292024/answer/47513658?utm_psn=2048893450939020688

搬运:FGwsz

要不要黑一下 C++呢? 呵呵呵。

咱们要有点娱乐精神, 关于 C++的笑话数都数不清

笑话: C++是一门不吉祥的语言, 据说波音公司之前用ADA为飞机硬件编程, 一直用的好好的, 后来招聘了一伙大学生, 学生们说我靠还在用这么落后的语言, 然后换成C++重构后飞机就坠毁了。

笑话: 什么是C++程序员呢? 就是本来10行写得完的程序, 他非要用30行来完成, 并自称“封装”, 但每每到第二个项目的时候却将80%打破重写, 并美其名曰“重构”。

笑话: C容易擦枪走火打到自己的脚, 用C++虽然不容易, 但一旦走火, 就会把你整条腿给炸飞了。

笑话: 同时学习两年 Java的程序员在一起讨论的是面向对象和设计模式, 而同时学习两年 C++的程序员, 在一起讨论的是 template和各种语言规范到底怎么回事。

笑话: 教别人学 C++的人都挣大钱了, 而很多真正用 C++的人, 都死的很惨。

笑话: C++有太多地方可以让一个人表现自己“很聪明”, 所以使用C++越久的人, 约觉得自己“很聪明”结果步入陷阱都不知道, 掉坑里了还觉得估计是自己没学好 C++。

笑话: 好多写了十多年 C++程序的人, 至今说不清楚 C++到底有多少规范, 至今仍然时不时的落入某些坑中。

笑话: 很多认为 C++方便跨平台的人, 实际编写跨平台代码时, 都会发现自己难找到两个支持相同标准的 C++编译器。

Q: 那 C++为什么还能看到那么多粉丝呢?

A: 其实是因为 Windows, 因为 Windows的兴起带动了 C++, C++本来就是一门只适合开发 GUI的语言。

Q: 为何 C++只适合开发 GUI呢?

A: 你看 Unix下没有 GUI, 为啥清一色的 C呀? 所有的系统级问题都能在 C里找到成熟的解决方案, 应用级问题都能用其他高级语言很好地解决, 哪里有 C++什么事情呀?

Q: 你强词夺理, Unix下也有 C++的项目呀。

A: 有, 没错, 你任然可以用任何语言编写任何糟糕的代码。

Q: 别瞎扯了, 你都在说些什么? 连C++和 Windows 都扯到一起去了。

A: 回想下当年的情景, 一个大牛在教一群初学者如何编程。一边开发一边指着屏幕上说, 你看, 这是一个 Button, 我们可以用一个对象来描述它, 那是一个 panel我们也可以用一个对象来描述它, 并且你们有没有发

现，其实 Button 和 Panel 是有血缘关系的，你们看。。这样就出来了。。。下面的学生以前都是学着学校落后的教材，有些甚至还在用 turboc 的 bgi 库来画一些点和圆。哪里见过这么这么华丽的 Windows 界面呀。大牛说的话，象金科玉律一样的铭刻在自己幼小的心理。一边学着 Windows，一边发现，果然，他们都需要一个基类，果然，他们是兄弟关系，共同包含一些基本属性，可以放到基类去。他们越用越爽，潜意识里觉得因为 C++ 这么顺利的帮他们解决那么多界面问题，那看来 C++ 可以帮他们解决一切问题了。于是开发完界面以后，他们继续开发，当他们碰到各种设计问题时，反而认为肯定自己没有用好 C++。于是强迫自己用下去，然后就完蛋了。

关于 C++ 的笑话我有一箩筐，各位 C++ 粉用不着对号入座。言归正传，为什么要黑 C++ 呢？谈不上黑不黑，我从 94 年开始使用 C++（先前是 C 和 Pascal），一路看着 C++ 成长壮大，用 C++ 写过的代码，加起来应该超过 10MB 了吧，C++ 的各种宝典我也都读过，一直到 2004 年开始切回 C，主要原因是发现很多没法用 C++ 思路继续解决下去的问题，或者说用 C++ 思路解决下去会很糟糕的问题。

那时候（2004-2005）正是 C++ 满天飞的时候，言必称 C++，用必用模版，我跳出来告诉你们醒醒吧，别过火了，这个世界并不是都是抽象数据结构和算法就可以描述清楚的。于是很多人激动的跳来说：“你没领会到 C++ 精髓，你根本都不会用 C++”。我问他们：“语言是用来解决问题的，如果一个语言学了三四年都会经常掉沟里，算好语言么？如果编写十多年 C++ 的程序员都很难掌握得了，这算好语言么”。他们又说：“语言是死的，人是活的”。

我记得当时一位国内 C++ 大牛，为了纠正我的“错误观点”，给我看过他写的一套十分强大的库，我打开一看，倒吸了一口冷气，全部是 .h 文件。我只能回他三个字：“你牛逼”。当然这是一个极端的例子，那家伙后来终于也开始把 .h 里面的东西逐步挪到 .cpp 里面了，这是好事。

当时和云风在一家公司，2004 年新人培训时，他给新人布置了一个实现内存分配器的作业，批改作业的时候，他经常边看边问人家，“不够 C++ 呀，你能不能百分之百 OOP？”，“1% 的 C 都不要留”。我当时在公司内部邮件列表里面发过关于 C++ 的问题，大部分人都表示：“你看没有 C++ 我们怎么写 3D 引擎呢？”。我跟他们讲：“John Carmack 直到 Quake3 都还在用着 ANSI C，后来因为不得不支持 D3D，改用 C++ 了。为啥 C 不能写 3D 引擎了？”。他们告诉我：“你看，Point，就是个对象，Matrix 也是个对象，那么多 Vector 的代数计算，用 C++ 的算术重载是多么美妙的事情，三维世界就是对象的世界。”。

确实当时客户端 GUI 的话，只有 C++，图形引擎也只有 C++，这两个正是 C++ 最强的地方，所以我也没和他们争辩，强迫他们承认 C 也可以很漂亮的写图形，而且 C 写的可以写的很优雅。我又不是闲着没事情，何必去质疑人家的核心价值观呢，呵呵。当年我正在接手一个 C++ 项目，代码超过 800KB，每次崩溃都需要花费很长时间去定位，项目中大量的前后依赖，改一个地方，前后要看好几处，一处遗漏，整个系统就傻逼了。我开始重构后，画了两个星期，将性能敏感的核心部分剥离出来用 C 实现（代码量仅 200KB），然后导出 Python 接口，用 Python 来完成剩下的部分，整个脚本层代码量只有 150KB。整个世界清爽了，整个 C++ 项目原来的工期为 2 个程序员四个月，我一个人重构的时间加起来就 1.5 个月，而且代码量比远来少了两倍还多，各种奇特的 BUG 也一扫而尽。我看看左边的 800KB 一团乱麻的 C++ 代码，再看看右边整洁的 300 多 KB 纯 C + Python，琢磨着，这个项目干嘛不一开始就这么做？

跨语言接口

现代项目开发，不但需要更高的性能，而且需要更强大的语言描述能力。而 C++ 正处在一个尴尬的地方，比底层，它不如 C 能够精确的控制内存和硬件，各种隐式构造让你防不胜防；比描述能力，比快速业务开发和错误定位，它又赶不上 Python, Ruby, Lua 等动态语言，处于东线和西线同时遭受挤压和蚕食的地步。很快，

2006-2007 年左右，其他项目组各种滥用 C++ 的问题开始显现出来：当时脚本化已经在工程实践中获得极大的成功，然而某些项目一方面又要追求 100% 的 C++，另一方面又需要对脚本导出接口，他们发现问题了，不知

道该怎么把大量的 C++ 基础库和接口导给 Lua。

C 的接口有各种方便的方式导给脚本，然而整个项目由一群从来就不消于使用脚本的 cpp 大牛开发出来，当我们要把 cpp 类导出接口给脚本时，他们设计了一套牛逼的系统，lua 自动生成机器码，去调用 c++ 的各种类，没错，就是 c++ 版本的 cffi 或者 ctypes。他为调用 vc 的类写了一套机器码生产，又为调用 gcc 的类写了一套代码生成。那位 cpp 大牛写完四处炫耀他的成果，后来他离职了，项目上线一而再再而三的出现无可查证的问题，后来云风去支援那个项目组，这套盘根错节的 c++ 项目，这套盘大的代码自生成系统深深的把他给恶心到了。后来众所周知云风开始反 C++，倡导回归 C 了，不知道是否和这个项目有关系。

于是发现个有趣的现象，但凡善于使用脚本来提高工程效率的人，基本都是 C 加动态语言解决大部分问题（除了 gui 和图形），但凡认为 c++ 统治宇宙的人很多都是从来没使用过脚本或者用了还不知道该怎样去用的人。

凭借这样的方法，我们的产品同竞争对手比拼时，同样一个功能，同样的人力配置，竞争对手用纯 C++ 要开发三月，我们一个月就弄出来了，同样的时间，对手只能试错一次，我们可以试错三次。后来，据我们招聘过来的同事说，竞争对手也开始逐步降低 C++ 的比例，增加 java 的比例了，这是好事，大家都在进步嘛。

ABI 的尴尬

ABI 级别的 C++ 接口从来没有标准化过，以类为接口会引入很多隐藏问题，比如内存问题，一个类在一个库里面实例化的，如果再另外一个库里面释放它们就有很多问题，因为两个动态库可能内存管理系统是不一样的。你用这里的 allocator 分配一块内存，又用那里的 allocator 去释放，不出问题才怪。很多解决方法是加一个 Release 方法（比如 DX），告诉外面的人，用完的时候不要去 delete，而是要调用 Release。项目写大了各个模块隔离成动态库是很正常的，而各种第三方库和自己写的库为追求高性能引入特定的内存管理机制也是很正常的。很多人不注意该调用 release 的地方错写成 delete 就掉沟里去了。更有甚者跨 ABI 定义了很多 inline 方法的类，结果各种隐式构造和析构其实在这个库里生成，那个库里被析构，乱成一团乱麻。C 就清晰很多，构造你就调用 fopen，析构你就 fclose，没有任何歧义。其实 C++ 的矛盾在于一方面承认作为系统级语言内存管理应该交给用户决定，一方面自己却又定义很多不受用户控制的内存操作行为。所以跨 ABI 层的 c++ 标准迟迟无法被定义出来，不是因为多态 abi 复杂，而是因为语言逻辑出现了相互矛盾。为了弥补这个矛盾，C++ 引入了 operator new，delete，这 new/delete 重载是一个补丁并没从逻辑上让语言变得完备，它的出现，进一步将使用者拖入 bug 的深渊。

其实今天我们回过头去看这个问题，能发现两个基本原则：跨 abi 的级别上引入不可控的内存机制从语言上是有问题的，只能要靠开发者约定各种灵巧的基类和约定开发规范来解决，这个问题在语言层是解决不了的；其次你既然定义了各种隐式构造和析构，就该像 java 活着动态语言一样彻底接管内存，不允许用户再自定义任何内存管理方法，而不是一方面作为系统级语言要给用户控制的自由，一方面自己又要抢着和用户一起控制。

因此对象层 ABI 接口迟迟无法标准化。而纯 C 的 ABI 不但可以轻松的跨动态库还能轻松的和汇编及各类语言融合，不是因为 C 设计多好，而是 C 作为系统层语言没有去管它不该管的东西。当年讨论到这个话题时 C++ 大牛们又开始重复那几句金科玉律来反驳我：“语言只是招式，你把内功练好，就能做到无招胜有招，拿起草来都可以当剑使，C++ 虽然有很多坑，你把设计做好不那么用不就行了”。我说：本来应该在语言层解决好的事情，由于语言逻辑不完备，将大量问题抛给开发者去解决极大的增加了开发者的思维负担，就像破屋上表浆糊一样。你金庸看多了吧，武术再高，当你拿到一把枪发现子弹不一定往前射，偶尔还会往后射时，请问你是该专心打敌人呢？还是时刻要提防自己的子弹射向自己？

系统层的挫败

C++ 遭受挫败是进军嵌入式和操作系统这样靠近硬件层的东西。大家觉得宇宙级别的编程语言，自然能够胜任一切任务，很快发现几个问题：

无法分配内存：原来用 C 可以完全不依赖内存分配，代码写几千行一个 malloc 没有都行。嵌入式下处理器加电后，跳到特定地址（比如起始地址 0），第一条指令一般用汇编来写，固定在 0 地址，就是简单初始化一下栈，然后跳转到 C 语言的 start 函数去，试想此时内存分配机制都还没有建立，你定义了两个类，怎么构造呀？资源有限的微处理器上大部分时候就是使用一块静态内存进行操作。C++ 写起来写爽了，各种隐式构造一出现，就傻了。

标准库依赖：在语言层面，C 语言的所有特性都可以不用依赖任何库就运行，这为编写系统层和跨平台跨语言代码带来了很方便的特性。而 C++ 就不行，我要构造呀，我要异常呀，你能不给我强大的运行时呢？什么你还想用 stl？不看看那套库有多臃肿呀（内存和代码尺寸）。

异常处理问题：底层开发需要严格的处理所有错误返回，这一行调用，下一行就判断错误。而异常是一种松散的错误处理方式，应用层这么写没问题，系统层这么写就很狼狈了。每行调用都 try 一下和 C 的调用后 if 判断结果有什么区别？C++ 的构造函数是没有返回值的，如果构造内部出错，就必须逼迫你 catch 构造函数的异常，即便你 catch 住了，此时这个实例是一个半初始化实例，你该怎么处理它呢？于是有人把初始化代码移除构造函数，构造时只初始化一下变量，新增加一个带返回的 init 函数，这样的代码写的比 C 冗余很多。何况硬件中断发生时，在你不知道的情况下，同事调到一些第三方的库，你最外层没有把新的 exception 给 catch 住，这个 exception 该往哪里抛呀？内存不够的时候你想抛出一个 OutOfMemoryException，可是内存已经不够了，此时完全无能力构造这个异常又该怎么办呢？

处理器兼容：C++ 的类依赖基地址+偏移地址的寻址方式，很多微处理器只有简单的给定地址寻址，不支持这样一条语句实现 BASE+OFFSET 的寻址，很多 C++ 代码编译出来需要更多的指令来运算地址，导致性能下降很多，得不偿失。

隐式操作问题：C 的特点是简单直接，每行语句你都能清楚的知道会被翻译成什么样子，系统会严格按照你的代码去执行。而用 C++，比如 `str1 = str2 + "Hello" + str3`；这样的语句，没几个人真的说得清楚究竟有多少次构造和拷贝，这样的写法编写底层代码是很不负责任的，底层需要更为精细和严格的控制，用 C 语言控制力更强。

当然，说道这里很多人又说，“C++ 本来就是 C 的超集，特定的地方你完全可以按照 C 的写法来做呀。没人强迫你构造类或者使用异常呀”，没错，按 Linus 的说法：“想要用 C++ 写出系统级的优秀的可移植和高效的代码，最终还是会限于使用 C 本身提供的功能，而这些功能 C 都已经完美提供了，所以系统层使用 C 的意义就在于在语言层排除 C++ 的其他特性的干扰”。

很多人都记得 Linus 在 2007 年因为有人问 Git 为什么不用 C++ 开发炮轰过一次 C++。事实上 2004 年 C++ 如日中天的时候，有人问 Linux 内核为何不用 C++ 开发，他就炮轰过一次了：

实际上，我们在 1992 年就尝试过在 Linux 使用 C++ 了。很恶心，相信我，用 C++ 写内核是一个 “BLOODY STUPID IDEA”。事实上，C++ 编译器不值得信赖，1992 年时它们更糟糕，而一些基本的事实从没改变过：

- 整套 C++ 异常处理系统是 “fundamentally broken”。特别对于编写内核而言。
- 任何语言或编译器喜欢在你背后隐藏行为（如内存分配）对于开发内核并不是一个好选择。
- 任然可以用 C 来编写面向对象代码（比如文件系统），而不需要用 C++ 写出一坨屎来。

总得来说，对任何希望用 C++ 来开发内核的人而言，他们都是在引入更多问题，无法象 C 一样清晰的看到自己到底在写什么。

C++ 粉丝们在 C++ 最火热的时候试图将 C++ 引入系统层开发，但是从来没有成功过。所以不管是嵌入式，还是操作系统，在靠近硬件底层的开发中，都是清一色的 C 代码，完全没有 C++ 的立足之地。

应用层的反思

STL出来后，给人一种 C++ 可以方便开发应用层逻辑的错觉。由于很多语言层不严密的事情，让 STL 来以补丁的方式完成，于是很多以为可以象写 java 一样写 C++ 的初学者落入了一个个的坑中。比如 `list.size()`，在 Windows 下 vc 的 stl 是保存了 list 的长度的，`size()` 直接 $O(1)$ 返回该变量，而在 gcc 的 stl 中，没有保存 list 长度，`size()` 将搜索所有节点， $O(n)$ 的速度返回。

由于语言层不支持字符串，导致 `std::string` 实现十分不统一，你拷贝构造一个字符串，有的实现是引用，才用 copy-on-write 的方法引用。有的地方又是 new，有的实现又是用的内存池，有的实现线程安全，有的实现线程不安全，你完全没法说出同一个语句后面到底做了些什么

见孟岩的《Linux之父话糙理不糙》：<https://blog.csdn.net/myan/article/details/1777230>

再比如说我想使用 `hash_map`，为了跨平台（当你真正编写跨平台代码时，你很难决定目标编译器和他们的版本，想用也用不了 `unordered_map`），我很难指出一种唯一声明 `hash_map` 的方法，为了保证在不同的编译器下正常的使用 `hash_map`，你不得不写成这样：

```
#ifdef __GNUC__
    #ifdef __DEPRECATED
        #undef __DEPRECATED
    #endif
    #include <ext/hash_map>
    namespace stdext { using namespace __gnu_cxx; }
    namespace __gnu_cxx {
        template<> struct hash< std::string > {
            size_t operator()( const std::string& x ) const {
                return hash< const char* >()( x.c_str() );
            }
        };
    }
#else
    #ifndef _MSC_VER
        #include <hash_map>
    #elif (_MSC_VER < 1300)
        #include <map>
        #define I HAVE NOT HASH_MAP
    #else
        #include <hash_map>
    #endif
#endif
#ifdef __GNUC__
    using namespace __gnu_cxx;
    typedef hash_map<uint32_t, XXXX*> HashXXXX;
#else
    using namespace stdext;
    typedef hash_map<uint32_t, XXXX*> HashXXXX;
#endif
```

如果有更好的跨平台写法，麻烦告诉我一下，实在是看不下去了。一个基础容器都让人用的那么辛苦，使得很多 C++ 程序员成天都在思考各种规范，没时间真正思考下程序设计。

由于语言层要兼容 C，又不肯象 C 一样只做好系统层的工作，导致当 C++ 涉足应用层时，没法接管内存管理，没法支持语言层字符串，没法实现语言层基础容器。所以需要借助一些 stl 之类的东西来提供便利，但 stl 本身又是充满各种坑的。且不说内存占用大，程序体积大等问题，当编译速度就够呛了。所以为什么 C++ 下面大家乐意重复造轮子，实现各种基本容器和字符串，导致几乎每个不同的 C++ 项目，都有自己特定的字符串实现。就是因为大家踩了坑了，才开始觉得需要自己来控制这些细节。stl 的出发点是好的，但是只能简单小程序里面随便用一下，真是大项目用，stl 就容易把人带沟里了，所以很多大点的 C++ 项目都是自己实现一套类似 STL 的东西，这难道不是违背了 stl 设计的初衷了么？

语言层的缺失，让大家为了满足业务开发的快速迭代的需求，创造了很多很基础的设计灵巧的基类，来提供类似垃圾回收，引用计数，copy-on-write，delegate，等数不胜数的功能。每个项目都有一系列 BaseObject 之类的基础类，这样就引入一个误区，两年后你再来看你的代码，发现某个 BaseObject 不满足需求了，或者你和另外一个项目 merge 代码时，需要合并一些根本属性。图形和 GUI 这些万年不变的模型还好，应用类开发千变万化，一旦这些设计灵巧的基类不再适应项目发展时，往往面临着全面调整的代价。

打开一个个 C++ 大牛们 blog，很多地方在教你 std::string 的原理，需要注意的事项。map 的限制，vector 的原理，教你如何实现一个 string。这就叫“心智负担”，分散你的注意力，这是其他语言里从来见不到的现象。战士不研究怎么上前线杀敌，天天在琢磨抢和炮的原理，成天在思考怎么用枪不会走火，用炮不会炸到自己，这战还怎么打？

所以此后几年，越来越多的人开始反思前两年 C++ 过热所带来的问题，

比如高性能网络库 ZeroMQ 作者 Martin Sustrik 的：《为什么我希望用 C 而不是 C++ 来实现 ZeroMQ》
<http://news.cnblogs.com/n/143021/>，

比如云风的《云风的 BLOG: C 的回归》https://blog.codingnow.com/2007/09/c_vs_cplusplus.html#more，

比如引起热议的《Why C++ Is Not "Back"》<http://simpleprogrammer.com/2012/12/01/why-c-is-not-back/>。

全面被代替

2008 年以后，行业竞争越来越激烈，正当大家一边苦恼如何提高开发效率，一边掉到 C++ 的各种坑里的时候，越来越多的应用开发方案涌现出来，他们都能很好的代替 C++。各行各业的开发者逐步相见恨晚的发现了各种更加优秀的方案：需要底层控制追求性能的设计，大家退回到 C；而需要快速迭代的东西大家找到各种动态语言；介于性能和开发速度之间的，有 java，知乎上好像很多黑 java 的，语言是有不足，但是比起 C++ 好很多，没那么坑，真正考虑面向对象，真正让人把心思放在设计上。所以再黑也不能挡住 java 在 tiobe 上和 C 语言不是第一就是第二的事实，再黑也挡不住 java 在云计算，分布式领域的卓越贡献。

所以 2005 年以后，C++ 处在一个全面被代替的过程中：

底层系统：进一步回归 C 语言，更强的控制力，更精确的操作。

网页开发：2006 年左右，C++ 和 fastcgi 就被一起赶出 web 世界了。

高性能服务：varnish, nginx, redis 等新的高性能网络服务器都是纯 C 开发的。

分布式应用：2007 年左右，C++ 被 java 和其他动态语言彻底赶跑。

游戏服务端：2008 年后进一步进化为 C 和 脚本，完全看不到胖 C++ 服务端了。

并行计算：2010 年后，go, scala, erlang；而能方便同 go 接口的，是 C 不是 C++。

游戏引擎：没错 C++ 和 脚本，但是这年头越来越多的开源引擎下，引擎类需求越来越少。

游戏逻辑：脚本

多媒体：SDL纯C，ffmpeg是纯C，webrtc的核心部分（DSP, codec）是纯C的。

移动开发：早年C++还可以开发下塞班，现在基本被 java + objc + swift 赶跑了。

桌面开发：Qt+Script, C#等都能做出漂亮的跨平台界面。且界面脚本化趋势，不需要C++了。

网页前端：JavaScript, Html5, Flash

操作系统：FreeBSD, Open Solaris, Linux, RTOS, Darwin（OS X 底层），都是纯C

虚拟技术：qemu / kvm（云计算的基石）纯C，Xen 纯C

数据库：MySQL (核心纯C，外围工具 C++)，SQLite 纯C, PostgreSQL / BDB / unqlite 纯C

编译器：C/C++并存，不过编译器用脚本写都没关系，我还在某平台用 java写的 C/C++编译器

大数据：kafka, hadoop, storm, spark 都使用 Java

云存储：openstack swift python, hdfs java, 还有好多方案用 go

可以看出，即便 C++的老本行，GUI和图形（确实也还存在一些短期内 C++无法替代的领域，就像交易系统里还有 COBOL一样），这年头也面临的越来越多的挑战。

比如新发布的 Rust (如何看待 Rust 的应用前景？ - 知乎用户的回答)

<https://www.zhihu.com/question/30407715/answer/48071161>

可以发现，开发技术多元化，用最适合的技术开发最适合的应用是未来的趋势。而为这些不同的技术编写高性能的可控的公共组件，并轻松和其他语言接口，正是 C语言的强项。所以不管应用层语言千变万化，对系统级开发语言C的需求还是那么的稳定，而在这个过程中，哪里还有 C++的影子呢？

话题总结

所以说未来的趋势是：C x 各种语言混搭 的趋势，从TIOBE上 C++的指数十年间下跌了三倍可以看出，未来还会涌现出更多技术来代替各个角落残存的C++方案，C++的使用情况还会进一步下降。所以题主问学习纯C是否有前途，我觉得如果题主能够左手熟练的掌握 C语言，培养系统化的思维习惯和精确控制内存和硬件的技巧；右手继续学习各种新兴的开发技术，能够应对各个细分领域的快速开发，碰到新问题时能左右开弓，那么未来工作上肯定是能上一个大台阶的。至于C++ 嘛，有时间看看就行，逼不得已要维护别人代码的情况下写两行即可。

故事分享

古代用弓箭进行远距离攻击时，对射手要求较高，瞄准难度大，需要一直使劲保持准心。战斗中一个弓箭手开弓二十次就需要比较长的休息时间。弩的威力远胜于弓，秦弩的制造就如现代的自动步枪一般精密无二，它既可以延长射击，又可以精确瞄准。弩箭的发射速度更是弓箭的数倍，威力惊人。因为弩的操作非常简单，不需要射击技巧，平民很容易掌握它的使用方法。秦国靠着弩兵，在战争中取得了不少优势，被人称为“虎狼之师”。日本投降时，天皇下罪己诏。很多士兵不愿意相信这时真的，找种种理由拒绝相信。有的士兵甚至以为天皇的广播是敌人诱降的把戏，于是躲到丛林里继续三五成群的收集情报，袭击可以攻击的目标，等待上司来给他们下达新命令。直到好几年后看到周围的人都穿着日常的便装了，而来巡山的“敌人”也从士兵变为了巡逻队，他们都还觉得这是敌人的伪装。而同时，德国战败时，最后的党卫军一直战斗到 1957年才肯投降。

很多人觉得 java 慢，C++ 快 java 10 倍以上已经是上世纪的事情了，现代的 java 只比 C/C++ 慢 70%，C++ 连 1 倍都快不了 java。也不要觉得动态语言慢，javascript 只比 C/C++ 慢 2.7 倍。lua jit 只比 C++ 慢 5.8 倍。在 jit 技术发展的今天，C++ 在性能上离动态语言/java 的差距越来越小，可易用性和生产效率上的差距，却和动态语言/java 比起来越来越大。

编辑于 2015-07-20 18:42